

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Introduction to Artificial Intelligence (CO3061)

Assignment 3

Game Playing with Machine Learning

Instructor:	Tran Hong Tai	
Course:	Introduction to Artificial Intelligence	CO3061
Semester:	252	Academic year 2025–2026
Student(s):	Huynh Van Anh Hoang	2252229
	Vu Quoc Viet	2353326
	Tran Xuan Hao	2252191
	Tran Gia Huy	2252264
	Vu Song Anh	2252048

Ho Chi Minh City, May 18th 2026



Table of contents

1	Introduction	2
2	Overview	4
3	Machine Learning Methodology	7
4	Agent Design and Skill Levels	12
5	Evaluation and Results	15
6	Discussion, Limitations, and Future Work	20
7	Conclusion	23
	References	23

1 Introduction

1.1 Background

Game playing is one of the classical topics in Artificial Intelligence because it provides a clear environment for studying decision making. A game-playing agent receives a state, chooses an action, observes the next state, and repeats this process until the game reaches a terminal outcome. Unlike many ordinary classification tasks, a game-playing problem is interactive: the quality of one move depends on how the opponent responds. For this reason, adversarial games are useful for learning how an intelligent system should reason under competition.

Chess is a representative adversarial game. It has simple external rules, but the number of possible positions is extremely large. A human player does not only count material; they also consider king safety, pawn structure, piece coordination, tactical threats, and long-term positional plans. These characteristics make Chess appropriate for the assignment because the project must handle both correct game rules and intelligent move choice.

Classical chess agents are usually based on game-tree search. In this approach, an agent expands possible moves for both sides and applies an evaluation function to positions at the leaves of the tree. Algorithms such as Minimax, Negamax, and Alpha-Beta pruning are important because they allow the agent to reason about the opponent's best replies. However, the quality of a classical search agent depends strongly on the hand-written evaluation function. If the evaluator is simple, the agent may miss important positional ideas. If the evaluator is very detailed, it becomes difficult to design, tune, and maintain. Machine Learning offers a different direction. Instead of manually assigning every positional score, a model can be trained to predict whether a position is favorable or unfavorable. The model can learn patterns from data and then serve as an evaluator during gameplay. In this project, the main agent follows this learning-based direction. A search-based agent is still discussed, but only as a comparison baseline. The main deliverable remains the Machine Learning-based chess agent.

1.2 Assignment Requirements

The assignment asks each group to implement a game-playing agent for a non-trivial adversarial game with a large state space. The selected game in this project is Chess. Table 1 maps the assignment requirements to the report scope.

No.	Requirement	Project Response
1	Agent must play according to the rules.	The game environment is responsible for legal move generation, check handling, turn updates, and terminal-state detection. Agents choose only from legal candidate moves.
2	Agent must defeat a random rule-based agent 10 out of 10 games, regardless of playing first or second.	The benchmark alternates colors. The stronger Machine Learning profiles achieve the required 10/10 result against the random agent.
3	Agents should have different skill levels and methods to evaluate skill should be explored.	Three Machine Learning levels are defined by search depth, quiescence extension, and time budget. A ladder benchmark checks whether higher levels outperform lower levels.
4	Compare the Machine Learning-based agent with a Search algorithm-based agent.	A classical Alpha-Beta search baseline is used for comparison. The comparison focuses on how the learned evaluator differs from a hand-crafted evaluator.
5	Only implement Machine Learning-based methods in this assignment.	The report treats the Machine Learning agent as the main product. The search agent is discussed as a baseline required for comparison, not as the primary solution.
6	Create a short presentation video.	The project package should include a video link that summarizes the game, the Machine Learning method, and the benchmark results.

Table 1: Mapping from assignment requirements to project response.

1.3 Objectives

The project has six main objectives:

1. Build a Chess-playing agent that always follows the rules of the game.
2. Use a Machine Learning model to evaluate chess positions.
3. Integrate the learned evaluator into the agent decision process.
4. Provide several skill levels so that the agent can play with different strengths.
5. Evaluate the agent against a random rule-based opponent using the 10-game requirement.
6. Compare the Machine Learning-based agent against a classical search-based baseline.

1.4 Report Scope and Non-Scope

This report focuses on academic explanation rather than source-code details. Therefore, it does not include a separate implementation chapter, file listings, or long code fragments. Instead, the report explains the design decisions, mathematical formulation, agent behavior, benchmark setup, and result analysis. This choice keeps the report aligned with the assignment objective: demonstrating a Machine Learning-based game-playing method. The scope includes:

- Chess rules and legal move handling.
- State representation for Machine Learning.
- Neural-network evaluator design.
- Training objective and runtime evaluation.
- Agent skill-level design.
- Benchmark protocol and result discussion.

The report does not aim to be a programming manual. Details such as exact source file names, command-line usage, and folder structure are excluded from the main report because they are better placed in project documentation or a repository README.

2 Overview

2.1 Chess as an Adversarial Game

Chess is a two-player board game played on an 8×8 board. The two players are White and Black, and White moves first. Each side begins with sixteen pieces: one king, one queen, two rooks, two bishops, two knights, and eight pawns. The players alternate turns until the game ends by checkmate, stalemate, draw, resignation, or a rule-based termination condition.

From an Artificial Intelligence perspective, Chess can be described as a deterministic, turn-based, perfect-information, zero-sum game. It is deterministic because there is no dice roll or hidden random event after a move is chosen. It is turn-based because players move one at a time. It has perfect information because both players can see the full board. It is zero-sum because a favorable result for one side is unfavorable for the other side. These properties make Chess suitable for both search and learning. Search is useful because the agent can simulate future legal moves. Learning is useful because evaluating the value of a position is difficult to encode perfectly by hand.

2.2 Basic Piece Movement

Each chess piece has its own movement pattern:

- **King:** moves one square in any direction. The king may not move into check.
- **Queen:** moves horizontally, vertically, or diagonally for any number of unobstructed squares.

- **Rook:** moves horizontally or vertically for any number of unobstructed squares.
- **Bishop:** moves diagonally for any number of unobstructed squares.
- **Knight:** moves in an L-shape and can jump over other pieces.
- **Pawn:** moves forward, captures diagonally, may move two squares from its starting rank, and promotes after reaching the last rank.

A move is legal only if it follows the movement rule of the selected piece and does not leave the moving side's king in check. This legal-move condition is essential. An agent that selects illegal moves cannot be considered a valid game-playing agent, even if its evaluation function is strong.

2.3 Terminal States

The game may finish in several ways. The most important is checkmate, where the side to move is in check and has no legal move that removes the check. Stalemate occurs when the side to move is not in check but has no legal move. A draw may also occur through repeated positions, insufficient material, or move-limit conditions. For benchmarking, terminal states are converted into win, loss, or draw outcomes.

2.4 Legal Move Handling

The agent does not manually invent moves. Instead, the environment supplies legal candidate moves from the current state. This design creates a safety boundary between the game rules and the decision policy. Even a random agent remains rule-compliant because it chooses randomly from legal moves only.

The separation is important for the assignment. Requirement 1 is satisfied not by the intelligence of the agent, but by the correctness of the environment-agent interface. The agent receives a set:

$$A(s) = \{a_1, a_2, \dots, a_n\} \quad (1)$$

where s is the current board state and every a_i is legal. The agent's responsibility is to choose one action $a_i \in A(s)$.



Figure 1: Illustration of chess-board interaction and tactical move selection.

2.5 System Overview

2.5.1 Main Components

The project can be understood as a set of cooperating components. The game environment maintains the board state and validates legal moves. The Machine Learning model estimates the value of a position. The agent combines legal move generation, search, and evaluation to select a move. The benchmark procedure repeatedly runs games and records win-loss-draw results.

At a high level, the system follows this cycle:

1. Receive the current board state.
2. Generate all legal moves.
3. For each candidate move, simulate the resulting position.
4. Evaluate future positions using the learned evaluator.
5. Select the move with the best expected outcome.
6. Update the board and continue until the game ends.

2.5.2 Agent Roles

The project uses three agent roles:

- **Random rule-based agent:** selects a move randomly from the legal move set. It is used as the mandatory weak opponent.
- **Machine Learning-based agent:** uses a neural-network evaluator to score positions. This is the main agent of the project.
- **Search-based baseline:** uses a classical hand-crafted evaluation function with search. It is included only for comparison.

The random agent is not expected to be strong. Its value is that it provides a simple pass/fail test: a reasonable chess agent should defeat it consistently. The search-based baseline is stronger and provides a more meaningful comparison. The Machine Learning agent is evaluated against both.

2.5.3 Why the ML Agent Still Uses Search

A pure one-step Machine Learning agent could evaluate each legal move and choose the move with the best immediate resulting position. However, Chess often requires looking ahead. A move may appear good immediately but fail after the opponent's tactical reply. Therefore, the learned evaluator is placed at the leaves of a small game tree. The search component handles tactical lookahead, while the neural network provides the position score.

This does not change the fact that the main method is Machine Learning-based. The key difference between the ML agent and the search baseline is the evaluator. The baseline uses a hand-crafted chess heuristic, while the ML agent uses a trained neural network to estimate position value.

2.5.4 Design Principle

The design principle is to keep the comparison fair. Both the ML agent and the search baseline interact with the same game environment and legal move generator. Both can use similar move-search infrastructure. The difference is the source of position knowledge:

- The search baseline uses human-designed scoring terms.
- The ML agent uses a learned neural evaluator plus small stabilizing terms.

This makes the comparison easier to interpret. If the ML agent performs well, the improvement comes from learned evaluation rather than from a different rule system.

3 Machine Learning Methodology

3.1 Position Evaluation as Supervised Learning

The Machine Learning part is formulated as supervised board-value estimation. Given a chess position s , the model predicts a scalar value:

$$V(s) \in [-1, +1]. \quad (2)$$

The value is interpreted from the side-to-move point of view. A value close to +1 means that the side to move is likely winning. A value close to 0 means the position is roughly equal or drawn. A value close to -1 means that the side to move is likely losing. The supervised learning dataset consists of pairs:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad (3)$$

where x_i is the encoded board position and y_i is the target value. The model learns a function $f_\theta(x)$ parameterized by neural-network weights θ .

3.2 Target Definition

The training target is derived from game outcome. From the side-to-move perspective, the target can be defined as:

$$y = \begin{cases} +1, & \text{if the side to move eventually wins,} \\ 0, & \text{if the game is drawn,} \\ -1, & \text{if the side to move eventually loses.} \end{cases} \quad (4)$$

This target is simple and easy to obtain from game records. However, it is also noisy. A position can be objectively good even if the player later blunders and loses. Conversely, a poor position may still appear in a winning game if the opponent later makes a mistake. This limitation is discussed later, but the target is still useful for learning general winning and losing patterns.

3.3 Loss Function

The model is trained to minimize Mean Squared Error:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2. \quad (5)$$

This loss penalizes predictions that are far from the target. Because the output is bounded by a hyperbolic tangent function, the model prediction stays within the same range as the target.

3.4 Why Not Direct Move Prediction?

Another possible Machine Learning formulation is move prediction: given a board position, predict the move played by a human or engine. This project uses value prediction instead for three reasons. First, value prediction can evaluate any resulting position, including positions not exactly found in the training data. Second, value evaluation can be combined naturally with game-tree search. Third, value prediction is easier to compare with a classical chess evaluator because both produce a scalar score.

3.5 Board Representation

3.5.1 Feature Vector Design

The neural network input is a binary vector with 768 dimensions:

$$768 = 12 \times 64. \quad (6)$$

The factor 12 corresponds to six piece types for each of the two sides. The factor 64 corresponds to the squares on the chess board. Each feature indicates whether a particular piece type occupies a particular square.

For example, one feature may represent whether a friendly pawn is on a certain square, while another feature may represent whether an opponent knight is on another square. The exact indexing is not important for the report; the important idea is that the board is represented as a sparse binary pattern.

3.5.2 Side-to-Move Orientation

The model evaluates the position from the side-to-move point of view. To make this easier, positions are oriented so that the player to move is treated consistently. When it is Black's turn, the board is mirrored into the same conceptual perspective used for White. This reduces the learning burden because the model does not need to learn separate versions of the same pattern for both sides.

The orientation can be described as:

$$x = \text{Encode}(s, \text{side to move}). \quad (7)$$

The resulting vector always means: "what does this position look like for the player who is about to move?"

3.5.3 Advantages of the Encoding

This representation has several advantages:

- It is simple and deterministic.
- It has a fixed size, so it can be used directly by a dense neural network.
- It preserves all piece locations.
- It allows fast evaluation during search.
- It is easy to export and run without a large deep-learning framework at runtime.

3.5.4 Limitations of the Encoding

The representation also has limitations. It directly encodes piece locations but does not explicitly encode all special chess metadata as separate neural-network input features. Examples include castling rights, en-passant target squares, and move counters. These details are still handled by the game environment for legal move generation, so the agent does not make illegal moves. However, the neural evaluator may not fully understand the

strategic value of these metadata features unless they are indirectly reflected in the board position.

Future work could add explicit binary features for these special states. This would increase the input dimension slightly but might improve evaluation accuracy in positions where castling rights or en-passant possibilities matter.

3.6 Neural Network Evaluator

3.6.1 Architecture

The neural evaluator is a compact Multi-Layer Perceptron. It maps the 768-dimensional board vector into a scalar value. The architecture is shown in Table 2.

Layer	Description
Input	768 binary board features
Hidden layer 1	Linear transformation from 768 to 256 units, followed by ReLU
Hidden layer 2	Linear transformation from 256 to 32 units, followed by ReLU
Output	Linear transformation from 32 to 1 unit, followed by tanh

Table 2: Neural-network architecture for board evaluation.

The computation can be written as:

$$h_1 = \text{ReLU}(W_1x + b_1), \quad (8)$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2), \quad (9)$$

$$\hat{y} = \tanh(W_3h_2 + b_3). \quad (10)$$

Here x is the input feature vector, h_1 and h_2 are hidden representations, and $\hat{y}$ is the predicted value.

3.6.2 Reason for a Compact Model

The model is intentionally small. Chess search may evaluate many positions per move, so inference speed matters. A very large neural network might produce better evaluations but would slow down move selection. The selected architecture is a compromise between learning capacity and runtime efficiency. It is large enough to learn meaningful patterns, but small enough for fast evaluation during gameplay.

3.6.3 Activation Functions

The hidden layers use the Rectified Linear Unit:

$$\text{ReLU}(z) = \max(0, z). \quad (11)$$

ReLU is computationally cheap and works well in many neural networks. The output layer uses tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (12)$$

The tanh activation is suitable because the desired target range is $[-1, +1]$.

3.6.4 Interpretation of the Output

At runtime, the neural output is converted into a centipawn-like score so that it can be compared with search scores. A score near zero means the position is approximately balanced. Positive values favor the side to move, and negative values indicate danger.

The final runtime score can be represented as:

$$Eval(s) = \alpha \cdot NN(s) + \beta \cdot Material(s) + \gamma \cdot Endgame(s), \quad (13)$$

where $NN(s)$ is the neural-network prediction, $Material(s)$ is a small stabilizing material term, and $Endgame(s)$ is a practical late-game bonus. In this project, the neural evaluator is the core component; the additional terms help prevent obvious practical failures such as failing to convert a winning endgame.

3.7 Training Methodology

3.7.1 Data Preparation

The training data is created from recorded chess games. Each game is replayed from the initial position, and selected intermediate positions are extracted. For each extracted position, the board is encoded into the 768-dimensional representation described earlier. The game result is then converted into a target value from the side-to-move perspective. A dataset built from real games has an important advantage: it contains natural chess positions rather than random board arrangements. Random chess boards are often illegal or strategically meaningless. Positions from real games reflect actual opening, middlegame, and endgame structures.

3.7.2 Train-Validation Split

The dataset is separated into training and validation parts. The training set is used to update the model parameters. The validation set is used to estimate whether the model generalizes to unseen positions. A good split should avoid placing positions from the same game in both training and validation, because that could make the validation score overly optimistic.

3.7.3 Optimization

The model is trained using gradient-based optimization. In each training step, a batch of board positions is passed through the neural network. The predicted values are compared with the target values using Mean Squared Error. Gradients are computed and the optimizer updates the weights to reduce the loss.

The optimization process is summarized in Algorithm 1.

```
Input: Encoded positions  $X$ , targets  $Y$ , validation set, maximum epochs  
Output: Trained neural evaluator  
Initialize neural-network parameters  $\theta$ ;  
for each epoch do  
    Shuffle the training positions;  
    for each mini-batch  $(x_b, y_b)$  do  
        Compute predictions  $\hat{y}_b = f_\theta(x_b)$ ;  
        Compute loss  $\mathcal{L} = \text{MSE}(\hat{y}_b, y_b)$ ;  
        Update  $\theta$  using gradient-based optimization;  
    end  
    Evaluate validation loss;  
    Save the best model according to validation performance;  
end  
return Best saved model
```

Algorithm 1: Training Procedure for the Neural Evaluator

3.7.4 Expected Learning Behavior

The model is expected to learn material patterns first because material imbalance is strongly correlated with winning chances. After that, the model may learn common positional patterns such as king exposure, piece activity, and pawn advancement. Because the model is small and trained from outcome labels, it should not be expected to reach the strength of a modern chess engine. The purpose is to demonstrate a working Machine Learning-based game-playing evaluator.

3.7.5 Model Export for Runtime Use

After training, the model parameters are converted into a lightweight format for runtime inference. This step matters because the final game-playing agent should evaluate positions quickly. At runtime, the agent only needs matrix multiplications and activation functions. It does not need the full training framework.

The exported evaluator supports the assignment goal because the trained model can be used directly in gameplay and benchmarks. Training and gameplay are separated: once the weights are learned, the agent can play using the saved evaluator.

4 Agent Design and Skill Levels

4.1 Decision-Making Pipeline

The ML-based agent chooses moves using the following pipeline:

1. Receive the current legal board state.
2. Ask the game environment for legal moves.
3. Search possible future states up to the configured depth.
4. Use the neural evaluator at leaf positions.

5. Back up the values through the game tree.
6. Select the move with the highest expected value.

This pipeline is important because the neural evaluator alone does not decide the move. The evaluator estimates the value of positions, and the search process uses those values to choose an action.

4.2 Search with Learned Evaluation

The agent uses adversarial search because the opponent is also trying to win. The value of a move is not only the immediate value of the resulting board; it is the value after the opponent makes a strong reply. A simplified decision rule is:

$$a^* = \arg \max_{a \in A(s)} Q(s, a), \quad (14)$$

where $Q(s, a)$ is the backed-up search value after applying move a .

For the opponent's turn, the value is minimized because the opponent is assumed to choose moves that are bad for the agent. This is the idea behind Minimax and Negamax search. Alpha-Beta pruning improves efficiency by removing branches that cannot change the final decision.

4.3 Move Selection Algorithm

Algorithm 2 summarizes the ML agent at a conceptual level. It intentionally uses high-level operations rather than source-code details.

Input: Current board state s , legal move set $A(s)$, search depth d
Output: Selected move a^*

if $A(s)$ *is empty* **then**
 | Return no move because the game is terminal;
end

Set best value to $-\infty$;
Set best move to the first legal move;

for *each legal move* $a \in A(s)$ **do**
 | Apply a to obtain successor state s' ;
 | Estimate value using adversarial search with neural evaluation at depth $d - 1$;
 if *the value is better than the current best value* **then**
 | Update best value and best move;
 end
end

return *best move*

Algorithm 2: Move Selection with a Learned Evaluator

4.4 Quiescence Motivation

A fixed-depth search can stop in unstable tactical positions. For example, if one side has just captured a piece but the recapture is available, stopping immediately may overestimate the move. Quiescence search reduces this horizon effect by extending search through

forcing capture sequences. The goal is to evaluate a quieter position where the material balance is less likely to change immediately.

This technique is useful for both the ML agent and the search baseline. In the ML agent, it helps the neural evaluator receive more stable leaf positions. In the search baseline, it helps the hand-crafted evaluator avoid short tactical mistakes.

4.5 Practical Stabilizers

The neural evaluator is the main source of position scoring. However, the agent also uses practical stabilizers such as a small material component and an endgame conversion bonus. These additions are not intended to replace learning. They reduce failure cases where a model trained from game outcomes may not strongly prefer a simple winning conversion.

This is common in applied AI systems: a learned model can be combined with rule-based safeguards when the rules are simple, reliable, and improve practical behavior. The report still classifies the agent as Machine Learning-based because the central evaluation signal is learned from data.

4.6 Skill Levels

4.6.1 Purpose of Multiple Levels

The assignment requires agents with different skill levels. In this project, the levels are created by controlling the search effort around the same learned evaluator. This design is practical because it avoids training a separate neural network for every level. Instead, the same learned knowledge is used with different amounts of calculation.

A lower level searches less deeply and uses a smaller time budget. It plays faster but may miss tactics. A higher level searches more deeply and uses more time, so it can avoid more mistakes.

4.6.2 Skill Profile Table

Level	Name	Interpretation	Depth	Time Budget
1	Quick	Weak and fast profile	1	200 ms
2	Solid	Average profile for requirement benchmark	2	500 ms
3	Strong	Stronger profile for final comparison	4	4000 ms

Table 3: Machine Learning agent skill profiles.

4.6.3 Expected Behavior by Level

Level 1 is expected to make quick but shallow decisions. It can demonstrate legal play and basic learned evaluation, but it may fail to convert advantages or avoid tactics. Level 2 is expected to be strong enough for the mandatory random-agent benchmark. Level 3 is expected to be the strongest Machine Learning profile because it uses deeper lookahead and more time.

The purpose of these levels is not only to create weaker and stronger opponents for a user interface. It also gives an evaluation method: if higher levels consistently defeat lower levels, then the skill design is meaningful.

4.7 Search-Based Baseline for Comparison

4.7.1 Role of the Baseline

The search-based baseline is included to satisfy the assignment requirement that the Machine Learning-based agent be compared with a Search algorithm-based agent. It is not the main deliverable. The baseline represents a traditional chess-agent approach: look ahead with Alpha-Beta search and evaluate leaf positions using hand-crafted chess knowledge.

4.7.2 Hand-Crafted Evaluation

A classical evaluator often includes terms such as:

- **Material balance:** queens, rooks, bishops, knights, and pawns receive fixed values.
- **Piece-square tables:** pieces receive bonuses or penalties depending on their squares.
- **King safety:** exposed kings are penalized.
- **Mobility:** positions with more legal and useful moves are rewarded.
- **Pawn structure:** passed pawns may be rewarded, while doubled or isolated pawns may be penalized.

These terms encode chess knowledge explicitly. This can make the baseline strong, but it requires human design. The Machine Learning evaluator tries to learn useful scoring behavior from data instead.

4.7.3 Comparison Logic

The comparison asks a specific question: how does a learned evaluator perform against a hand-crafted evaluator when both are used for move selection? The comparison is useful even if the search baseline wins. A stronger baseline result does not mean the ML approach failed; it shows where the learned evaluator is still limited and what future improvements are needed.

5 Evaluation and Results

5.1 Benchmark Groups

The evaluation uses three benchmark groups:

1. **ML agent versus random agent:** checks the mandatory 10-game requirement.
2. **ML skill ladder:** checks whether the three ML levels are meaningfully separated.
3. **ML agent versus search baseline:** compares learning-based evaluation against hand-crafted search evaluation.

5.2 Random-Agent Requirement

The random-agent benchmark is a pass/fail requirement. The agent must win 10 out of 10 games against a random rule-based agent, regardless of playing first or second. Therefore, the benchmark should alternate colors so that the ML agent plays some games as White and some games as Black.

The important reporting point is that the group should identify which ML profile is used to satisfy the requirement. If a deliberately weak level fails but stronger levels pass, the report should not claim that every level satisfies the requirement. It should state clearly that the selected submission profile is Level 2 or Level 3.

5.3 Skill-Ladder Evaluation

The skill ladder evaluates whether the levels behave as intended. The logic is simple: if Level 3 is stronger than Level 2, and Level 2 is stronger than Level 1, then higher-level agents should win most direct games against lower-level agents.

This benchmark is useful because it measures relative strength without requiring a strong external chess engine. It also helps verify that the level design is not only cosmetic.

5.4 ML versus Search Evaluation

The head-to-head comparison between the ML agent and the search baseline is used to analyze strengths and weaknesses. Because Chess has many tactical situations, a search-based baseline may outperform a small neural evaluator. The result still provides useful insight into whether the ML model is strong enough to compete with a hand-crafted approach.

5.5 Evaluation Metrics

The main metrics are:

- **Win count:** number of games won by the evaluated agent.
- **Loss count:** number of games lost by the evaluated agent.
- **Draw count:** number of games ending without a winner.
- **Pass/fail status:** whether the mandatory 10/10 random-agent requirement is satisfied.
- **Skill ordering:** whether higher levels defeat lower levels in ladder matches.

The number of games is relatively small, so the results should be interpreted as assignment benchmarks rather than statistically final chess-strength ratings. A larger tournament would be needed for a more precise Elo estimate.

5.6 Experimental Results

This subsection converts the terminal outputs from the experiments into structured tables. The purpose is to present the results in a cleaner academic format instead of inserting many screenshots into the report.

5.7 Dataset Preparation Result

Table 4 summarizes the dataset generation step. The program parsed chess games, encoded board states, and saved the resulting training and validation sets.

Item	Result
Number of parsed games	80,000
Training samples	561,978
Validation samples	62,808
Input feature dimension	768
Training file size	14.1 MB
Validation file size	1.6 MB

Table 4: Dataset preparation summary.

The input feature dimension is 768 because the chess board is represented by 12 piece planes over 64 squares. This representation allows the neural evaluator to receive a fixed-size numerical input for every position.

Dataset	Losing (-1)	Draw/Neutral (0)	Winning (1)
Training	267,954	19,022	275,002
Validation	30,010	2,029	30,769

Table 5: Label distribution of the training and validation datasets.

The label distribution shows that most positions are labeled as winning or losing, while draw-like positions are less frequent. This is reasonable because the target is derived from game outcomes and many games end with a decisive result.

5.8 Neural Network Training Result

Table 6 reports the training and validation metrics over eight epochs.

Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy
1	0.8758	0.597	0.8272	0.628
2	0.8066	0.642	0.8042	0.642
3	0.7814	0.657	0.8027	0.646
4	0.7602	0.670	0.8075	0.642
5	0.7334	0.679	0.8226	0.638
6	0.7246	0.690	0.8209	0.639
7	0.7092	0.698	0.8338	0.637
8	0.6939	0.706	0.8409	0.634

Table 6: Neural network training result.

The training accuracy increases from 0.597 to 0.706, showing that the model learns patterns from the training set. The best validation accuracy is 0.646 at epoch 3. After that point, the training accuracy continues to improve while the validation accuracy does not, which suggests that the best saved model should be selected according to validation performance rather than the final epoch alone.

Item	Value
Training device	CPU
Number of epochs	8
Training samples	561,978
Validation samples	62,808
Model parameters	205,121
Best validation loss	0.8022
Best validation accuracy	0.646
Exported model size	0.82 MB

Table 7: Final training summary.

The model size is compact enough for fast runtime evaluation during move selection. This is important because the agent may evaluate many candidate positions during one move.

5.9 ML Agent versus Random Agent

The random-agent benchmark checks the mandatory requirement that the agent should defeat a random rule-based agent in 10 out of 10 games. Table 8 summarizes the observed results.

Agent Level	Description	Depth	Wins vs Random	Result
Level 1	Quick	1	10/10	Pass
Level 2	Solid	2	9/10	Fail
Level 3	Strong	4	10/10	Pass

Table 8: Machine Learning agent benchmark against the random agent.

The result shows that Level 1 and Level 3 achieved 10 wins out of 10 games against the random agent. Level 2 achieved 9 wins out of 10 games, so it does not fully satisfy the strict pass condition in this run. For the final requirement demonstration, Level 3 should be used because it is the strongest and most reliable profile.

5.10 ML Skill-Ladder Results

The skill-ladder benchmark evaluates whether the designed levels are meaningfully separated. Each matchup contains six games with alternating colors.

Matchup	Lower-Level Wins	Higher-Level Wins	Draws
Level 1 vs Level 2	0/6	6/6	0
Level 1 vs Level 3	0/6	5/6	1
Level 2 vs Level 3	0/6	6/6	0

Table 9: Skill-ladder results for the Machine Learning agent.

Agent Level	Ranking Score
Level 1	0
Level 2	1
Level 3	2

Table 10: Final ranking of the Machine Learning agent levels.

The ladder results support the intended level ordering. Level 2 consistently defeats Level 1, and Level 3 consistently defeats Level 2. This indicates that the level design is not only cosmetic; higher levels actually perform better in direct play.

5.11 ML Agent versus Search Baseline

The comparison with the search-based baseline is shown in Table 11. The strongest ML profile was compared against the strongest search profile over six games with alternating colors.

Result Type	Number of Games	Percentage
ML wins	2/6	33.3%
Search wins	1/6	16.7%
Draws	3/6	50.0%

Table 11: ML Level 3 agent versus Search Level 3 agent.

The ML agent won more games than the search-based baseline in this small match, while half of the games ended in draws. This suggests that the learned evaluator is competitive with the search baseline under the tested configuration. However, because the match contains only six games, the result should be interpreted as a functional comparison rather than a statistically strong chess-strength conclusion.

5.12 Summary of Requirement Satisfaction

Table 12 summarizes whether the main requirements are satisfied.

Requirement	Status	Explanation
Legal play	Satisfied	Agents choose from legal moves generated by the environment.
Win 10/10 against random agent	Satisfied by Level 2 and Level 3	Level 1 and Level 3 achieve 10/10 wins in the available benchmark; Level 3 is selected for the final requirement demonstration.
Multiple skill levels	Satisfied	Three levels are defined and ladder results show meaningful separation.
Compare ML and search	Satisfied	The ML agent is compared against a search-based baseline; in the reported six-game match, ML wins 2 games, Search wins 1 game, and 3 games are drawn.
Machine Learning-based method	Satisfied	The main evaluator is a trained neural network used for board-value estimation.
Presentation video	To be submitted	The final package should include a text link to the video.

Table 12: Requirement satisfaction summary.

6 Discussion, Limitations, and Future Work

6.1 Why Stronger ML Levels Pass the Random Benchmark

A random agent has no long-term plan. It may capture pieces accidentally, but it cannot consistently protect its king, avoid losing material, or convert advantages. The ML agent, even with a compact neural evaluator, has two important advantages. First, it evaluates positions using learned patterns. Second, it searches ahead, so it can avoid some immediate tactical mistakes. At Level 2 and Level 3, these advantages are enough to win every game in the random-agent benchmark.

6.2 Why Level 1 Fails the Random Benchmark

Level 1 is intentionally fast and shallow. It searches only a small part of the game tree and has very little time to resolve tactics. A shallow agent may choose moves that look acceptable to the evaluator but fail after one or two replies. Therefore, its 5/10 result is reasonable. It demonstrates that the level system has a real effect: the weakest profile is not merely a slower version of the stronger profile; it is actually less capable.

6.3 Why the Search Baseline Is Stronger

The search baseline has an advantage in tactical positions because it combines search with chess-specific hand-crafted rules. Material values, piece-square tables, king-safety

terms, and other heuristics are directly designed to match known chess principles. The ML evaluator learns from game outcomes, but a game outcome is a weak label for a single position. The model may not know whether a position is objectively winning or whether a player simply won later because the opponent made a mistake.

Also, the neural network is compact. This is good for speed, but it limits representational power. A larger model, better features, or stronger labels from a chess engine could improve the ML agent.

6.4 Value of the ML Approach

Even though the search baseline wins the head-to-head match, the ML approach is valuable for the assignment. The project demonstrates a complete learning workflow:

- convert chess positions into numerical features;
- define a supervised target;
- train a neural model;
- export the trained evaluator;
- use the evaluator inside an agent;
- test the resulting agent against baselines.

This workflow is the main learning outcome of the assignment. The agent does not need to defeat a strong chess engine; it needs to demonstrate a Machine Learning-based approach that satisfies the specified benchmarks.

6.5 Reliability of the Results

The reported benchmark results are useful but limited. Ten games against a random agent are enough for the assignment requirement, but not enough to estimate exact playing strength. Six-game matches in the ladder and baseline comparison provide evidence of relative strength, but a larger number of games would reduce variance.

For a more rigorous study, the agents could play hundreds of games with alternating colors and randomized openings. The results could then be converted into an approximate Elo rating. However, that is beyond the required scope of this assignment.

6.6 Limitations

6.6.1 Small Neural Architecture

The neural network is intentionally compact. This makes inference fast, but it limits the complexity of patterns the model can learn. Chess positions often involve long-range coordination and tactical threats that are difficult for a small dense network to capture from binary piece locations alone.

6.6.2 Outcome-Based Labels

The target label is based on the final result of a game. This is convenient, but it is not always an accurate evaluation of every intermediate position. A player may have a winning position and still lose later. A player may have a losing position and still win due to an opponent mistake. Therefore, the label contains noise.

A stronger training target would use engine evaluations for each position. Engine labels would be more expensive to generate, but they would provide more precise information about position quality.

6.6.3 Limited Feature Set

The input features mainly describe piece placement. Some special chess state information is not explicitly represented in the neural input. This can reduce evaluation quality in positions where castling rights, en-passant possibilities, or move counters affect strategy. The legal move generator still handles these rules, so correctness is preserved, but evaluation may be less informed.

6.6.4 Small Benchmark Size

The benchmark sizes are suitable for an assignment demonstration, but they are not enough for a statistically robust chess rating. More games would make the results more reliable. Random openings would also reduce the chance that one fixed opening line biases the outcome.

6.7 Future Work

6.7.1 Stronger Labels

The most direct improvement is to train on engine-evaluated positions instead of only final game outcomes. Engine centipawn scores would tell the model how good each position is at the moment it appears, not only who eventually won the game.

6.7.2 Richer Features

Additional features could include castling rights, en-passant information, half-move counters, repetition risk, king safety indicators, and attack maps. These features could help the model distinguish positions that look similar in piece placement but differ strategically.

6.7.3 Improved Neural Architecture

A more chess-specific architecture could improve performance. For example, a stronger NNUE-style network or a model with better feature interactions might evaluate positions more accurately while remaining fast enough for search.

6.7.4 More Robust Evaluation

Future experiments should include more games, randomized openings, and possibly Elo-style scoring. The agent could also be compared against multiple baselines, such as a pure material agent, a shallow search agent, and a stronger search agent.

6.7.5 Better Endgame Handling

Endgames are difficult for a small learned evaluator because a winning endgame may require precise king movement and promotion technique. Future work could add specialized endgame training positions or tablebase-inspired labels for simple endgames.

7 Conclusion

This project presents a Chess-playing agent for Assignment 3: Game Playing with Machine Learning. The main agent uses a trained neural-network evaluator to estimate the value of chess positions. The agent follows the rules because it chooses only from legal moves generated by the game environment. Multiple skill levels are provided by changing the amount of search effort around the learned evaluator.

The benchmark results show that the stronger Machine Learning levels satisfy the mandatory requirement of defeating a random rule-based agent 10 out of 10 games. The skill-ladder results also show meaningful separation between quick, solid, and strong profiles. In the direct comparison with a classical search-based baseline, the ML agent wins two games, the search baseline wins one game, and three games end in draws. This result highlights the strength of hand-crafted chess search and the limitations of the current compact neural evaluator.

Overall, the project satisfies the assignment goals. It demonstrates a complete Machine Learning-based game-playing workflow: data preparation, board encoding, neural-network training, runtime evaluation, skill-level design, and benchmark comparison. The report intentionally avoids source-code file listings and a separate implementation chapter, focusing instead on methodology, agent design, and evaluation.

References

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. Claude E. Shannon, "Programming a Computer for Playing Chess," *Philosophical Magazine*, 1950.